

AWS VPC — Networking Deep Dive & Production Architecture

Module: AWS Networking | **Level:** Beginner to Production **Goal:** Build a solid understanding of VPC concepts, then deploy a complete three-tier production architecture on AWS.

Table of Contents

1. [What is a VPC?](#)
2. [Public Subnet](#)
3. [Private Subnet](#)
4. [Internet Gateway](#)
5. [NAT Gateway](#)
6. [Bastion Host](#)
7. [Route Tables](#)
8. [Security Groups](#)
9. [Production Task — Three-Tier Architecture](#)

1. What is a VPC?

◆ Definition

A **Virtual Private Cloud (VPC)** is a logically isolated network that you create inside the AWS Cloud. It gives you complete control over your own networking environment — IP ranges, subnets, routing rules, and security settings — all within AWS infrastructure.

A simple way to think about it: imagine renting an entire floor of a shared office building. The building is AWS, but your floor is completely separated from everyone else. You decide where the walls go, which rooms connect to each other, and who gets a key to the front door.

When you create an AWS account, AWS automatically creates a **Default VPC** in every region. However, for real production workloads, you always create a **Custom VPC** where you define everything yourself.

◆ How AWS Handles It

- You assign your VPC a **CIDR block** — a range of private IP addresses your resources will use

- Everything inside stays **completely private by default** — no internet access unless you set it up
- A VPC is **region-scoped** — it spans across all Availability Zones in that region automatically
- You can run multiple VPCs per region (default limit is 5, which can be raised)
- VPCs can be connected to each other using **VPC Peering** or **AWS Transit Gateway**

◆ Why VPC Matters

Without a Custom VPC	With a Custom VPC
Shared address space with defaults	Full control over IP address design
Default routing you can't fully customize	Custom route tables per subnet
Limited security segmentation	Multiple security layers (SGs, NACLs)
Not suitable for enterprise workloads	Production-ready, audit-friendly setup

◆ Understanding CIDR Blocks

CIDR (Classless Inter-Domain Routing) defines how large your IP address pool is.

VPC CIDR: 10.0.0.0/16

Breaking it down:

/16 = first 16 bits are fixed (the "10.0" part)

Remaining 16 bits are yours to assign

Total available IPs = 65,536

You slice **this** into smaller subnets:

10.0.1.0/24 → 256 IPs → Web Subnet AZ-1a

10.0.2.0/24 → 256 IPs → Web Subnet AZ-1b

10.0.3.0/24 → 256 IPs → App Subnet AZ-1a

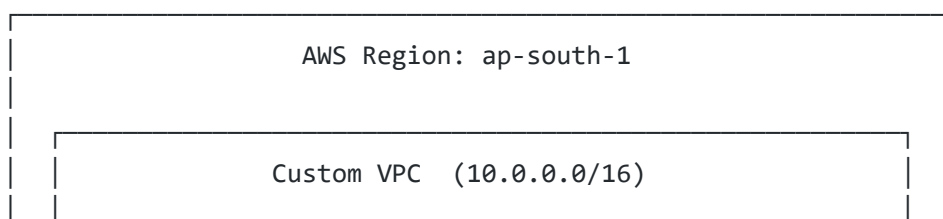
10.0.4.0/24 → 256 IPs → App Subnet AZ-1b

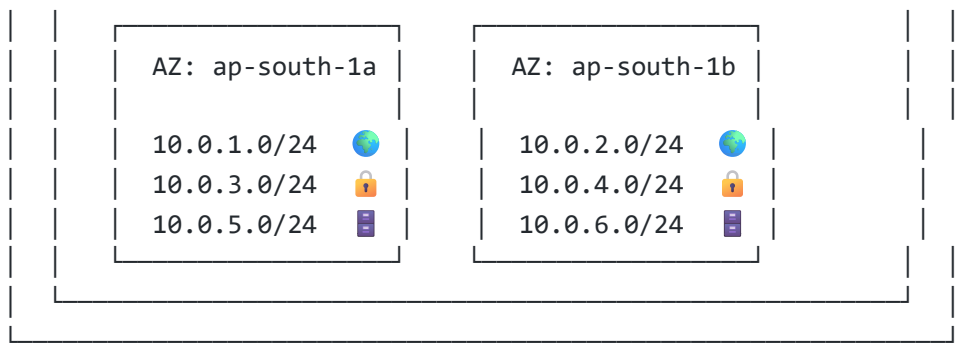
10.0.5.0/24 → 256 IPs → DB Subnet AZ-1a

10.0.6.0/24 → 256 IPs → DB Subnet AZ-1b

Starting with a /16 gives you plenty of room to grow without having to redesign your network later.

◆ VPC Layout Diagram





= Public Subnet
 = Private App Subnet
 = DB Subnet

2. Public Subnet

Definition

A **Public Subnet** is a subnet that has a direct route to the Internet Gateway in its route table. Resources placed here can be assigned public IPv4 addresses and can communicate both inward and outward with the internet.

Think of a public subnet like the lobby of an office building — it's the part that faces the street and is accessible to anyone coming in from outside.

Defining Characteristics

- Route table contains: `0.0.0.0/0 → Internet Gateway`
- Instances can receive **public IPv4 addresses** (auto-assign setting must be enabled)
- Traffic can flow in **both directions** — from the internet to the resource and back
- Used only for resources that genuinely **need to be reachable from outside**

What Belongs in a Public Subnet?

Resource	Reason It Needs Public Access
Application Load Balancer (ALB)	Receives HTTP/HTTPS traffic from internet users
NAT Gateway	Must reach the internet to forward private instance traffic
Bastion Host	Needs to be SSH-accessible from developer machines
Web-facing NGINX	Serves frontend content directly to browsers

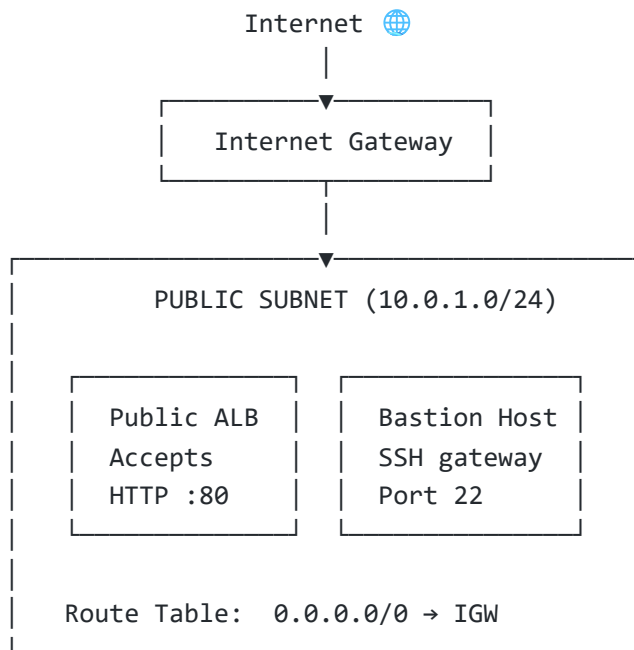
Important AWS Detail — Reserved IPs

AWS reserves **5 IP addresses** in every subnet. For a `/24` subnet (256 total):

10.0.1.0 → Network address (reserved)
10.0.1.1 → VPC Router (AWS internal)
10.0.1.2 → AWS DNS server
10.0.1.3 → Reserved for future AWS use
10.0.1.255 → Broadcast address (reserved)

Usable IPs: $256 - 5 = 251$

Public Subnet Traffic Flow



3. Private Subnet

Definition

A **Private Subnet** is a subnet with no direct route to the Internet Gateway. Instances here only receive private IP addresses and are completely unreachable from the public internet. Any outbound internet traffic must flow through a **NAT Gateway** placed in a public subnet.

Using the office building analogy — private subnets are the inner offices and server rooms. Nobody from outside can walk directly in. Visitors must go through the lobby (public subnet) first.

Why Private Subnets Are Critical

The security principle is simple: **if attackers cannot reach your servers, they cannot attack them.** Keeping your application logic and database in private subnets eliminates an entire category of attack vectors.

Even if your web tier gets compromised, the attacker still cannot directly access your database because it has no public IP and no route to the internet from outside.

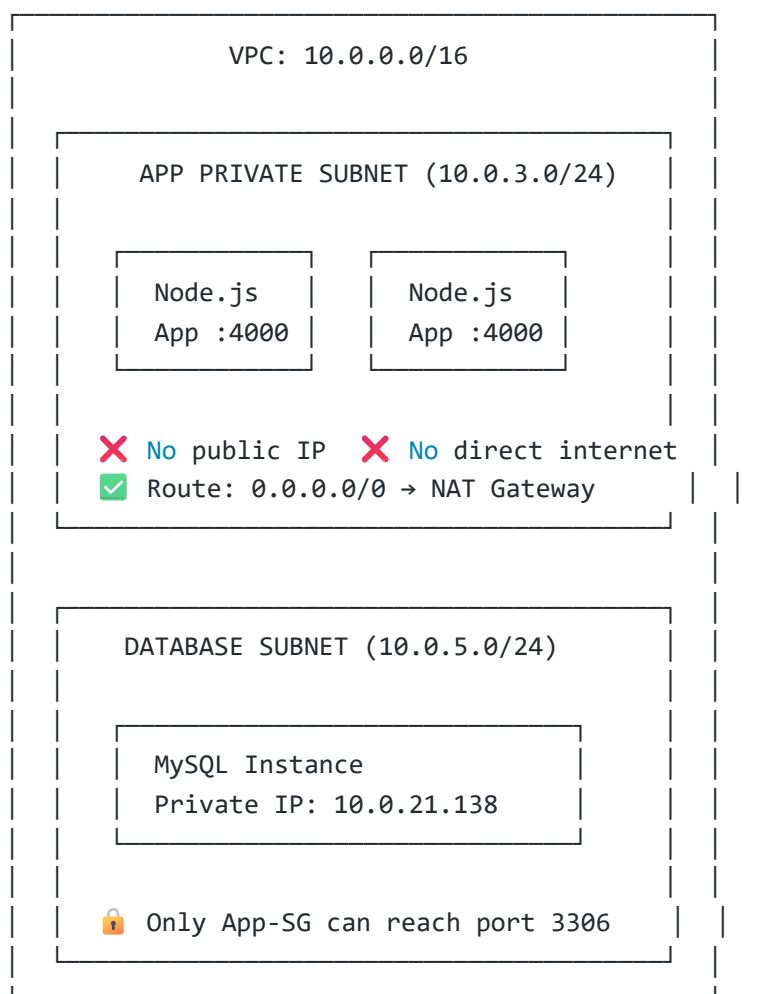
Private Subnet Categories

Subnet Type	Hosts	Example CIDR
 App Private Subnet	Node.js / backend API servers	10.0.3.0/24, 10.0.4.0/24
 Database Private Subnet	MySQL / RDS / database instances	10.0.5.0/24, 10.0.6.0/24

Private Subnet Networking Rules

- No route to Internet Gateway — completely inbound-blocked from internet
- Outbound internet (for OS updates, package installs) goes through **NAT Gateway**
- Inbound connections only accepted from within the VPC — other subnets, load balancers, bastion host
- Database subnets are typically even more restricted — only the App tier can connect

Private Subnet Diagram



4. Internet Gateway

Definition

An **Internet Gateway (IGW)** is a managed AWS component that you attach to your VPC to enable internet connectivity. It performs two specific functions:

1. Acts as a **routing target** in route tables for internet-bound traffic
2. Handles **Network Address Translation (NAT)** — translating the public IP of an instance to its private IP and back

Without an IGW attached to your VPC, no traffic can flow between your VPC and the public internet — even if instances have public IPs assigned.

Key Facts

- **One per VPC** — you can only attach a single IGW to a VPC at a time
- **Fully managed** — AWS handles all availability, capacity, and scaling automatically
- **No bandwidth cap** — it scales to handle any traffic volume without configuration
- **No cost to create** — you only pay for data transfer through it
- Must be explicitly **attached** to a VPC after creation — creating it alone does nothing

How Public IP Translation Works

Scenario: User accesses your website

User IP: 203.0.113.55

|



Hits IGW → IGW translates destination **to** instance private IP

|



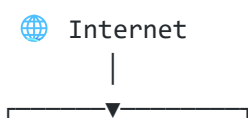
Instance Private IP: 10.0.1.25

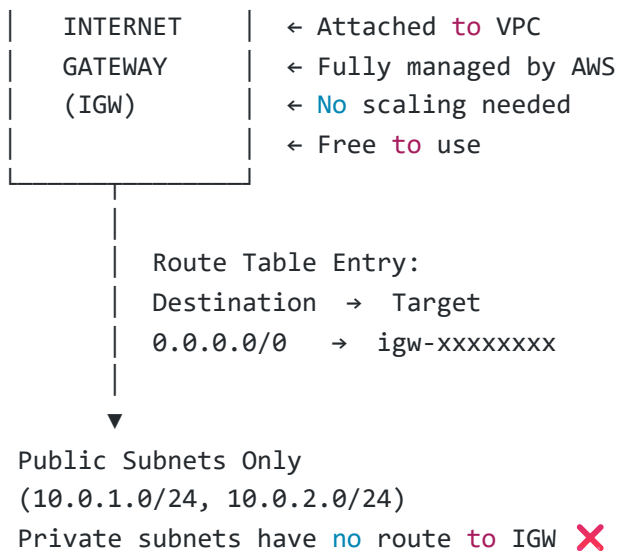
(Instance has a Public IP mapped **to** it by AWS)

Return path: same translation **in** reverse

10.0.1.25 → IGW → 203.0.113.55

IGW Architecture Diagram





🌐 IGW vs NAT Gateway Comparison

Feature	Internet Gateway	NAT Gateway
Traffic Direction	Inbound + Outbound	Outbound only
Used By	Public subnets	Private subnets
Cost	Free	Paid (hourly + data GB)
Requires Public IP on instance?	Yes	No (NAT has its own Elastic IP)
Managed by AWS?	Yes	Yes

5. NAT Gateway

🔄 Definition

A **NAT Gateway (Network Address Translation Gateway)** is a managed AWS service that allows instances in private subnets to initiate outbound connections to the internet while ensuring the internet cannot initiate connections back to those instances.

It works by replacing the private IP of your instance with its own public Elastic IP when sending traffic outward. From the internet's perspective, all traffic appears to come from the NAT Gateway — your private instances remain completely hidden.

🔄 Why Private Instances Still Need Internet Access

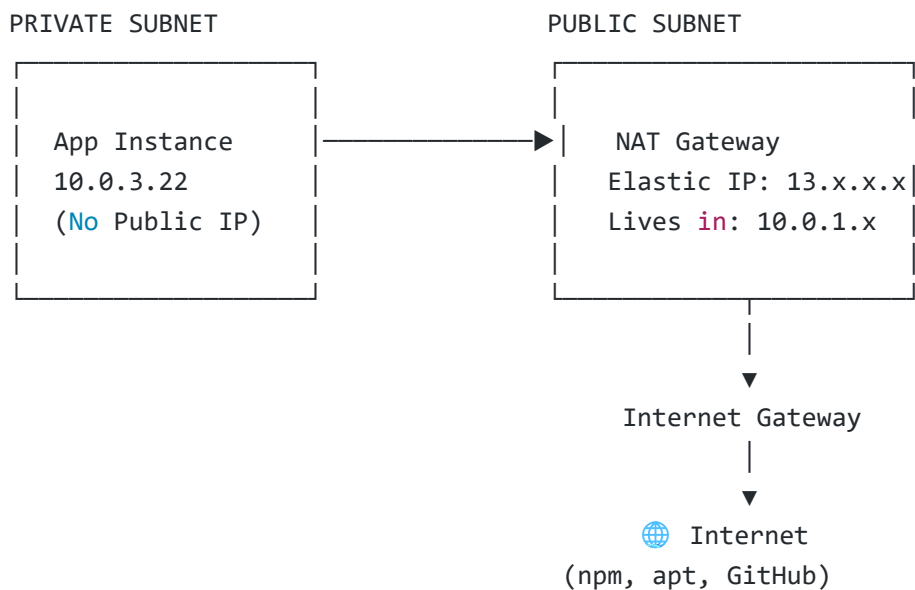
Even though your app servers and databases are private, they still need to reach the internet for operational reasons:

Common outbound needs from private instances:

- |— `sudo apt update` → Download OS **security** patches
- |— `npm install` → Pull Node.js packages **from** npmjs.com
- |— `git clone` → **Fetch** application code **from** GitHub
- |— `pip install` → Python **package** downloads
- |— API calls → Third-party services (Stripe, SendGrid, etc.)

NAT Gateway handles all of this while keeping the instances invisible to inbound internet traffic.

NAT Gateway Traffic Flow



Response path:

Internet → IGW → NAT Gateway → App Instance ✓

Important Deployment Notes

- NAT Gateway **must be placed in a public subnet** — it needs internet access itself
- Each NAT Gateway lives in **one Availability Zone** — for multi-AZ high availability, create one NAT Gateway per AZ
- Assign it an **Elastic IP** (static public IP) during creation
- Private route tables must point `0.0.0.0/0` to the NAT Gateway

Multi-AZ NAT Setup (Recommended **for** Production):

AZ-1a: NAT-GW-1 (EIP: 13.0.0.1) ← used by private subnets **in** 1a

AZ-1b: NAT-GW-2 (EIP: 13.0.0.2) ← used by private subnets **in** 1b

If AZ-1a fails, private instances **in** 1b still have internet access ✓

Single NAT setup: **if** that AZ fails, ALL private internet access breaks ✗

NAT Gateway vs NAT Instance

Feature	NAT Gateway 	NAT Instance 
Management	Fully AWS managed	You manage the EC2
Availability	High availability within AZ	Single point of failure
Max Bandwidth	Up to 45 Gbps	Limited by EC2 type
Setup Effort	Minimal	Requires manual config
Cost	Higher	Cheaper (EC2 pricing)
Recommended For	All production workloads	Dev/lab environments only

6. Bastion Host

Definition

A **Bastion Host** (sometimes called a Jump Server or Jump Box) is a dedicated EC2 instance placed in a public subnet that acts as a secure, controlled entry point for SSH access to instances in private subnets.

Because your app and database servers live in private subnets with no public IP, you cannot SSH into them directly. The Bastion Host bridges this gap — you SSH into the Bastion first, then SSH onward to your private instance from there.

The Security Problem It Solves

Without Bastion Host:

Developer Laptop → ??? → Private App Server

No route exists. Cannot connect. 

With Bastion Host:

Developer Laptop → Bastion Host → Private App Server 

Controlled, logged, single entry point.

Security Best Practices

Practice	Why It Matters
Restrict SSH source IP to your own IP	Prevents the entire internet from attempting logins

Practice	Why It Matters
Always use key pairs, never passwords	Keys are cryptographically stronger than passwords
Enable AWS CloudTrail logging	Creates an audit trail of who logged in and when
Stop the Bastion when not in use	Reduces attack window and saves EC2 cost
Consider AWS Systems Manager Session Manager	Zero-bastion alternative — no open port 22 needed

Connecting Through the Bastion

Method 1 – Two separate SSH hops

Hop 1: Connect to Bastion (it has a public IP)

```
ssh -i mykey.pem ubuntu@<bastion-public-ip>
```

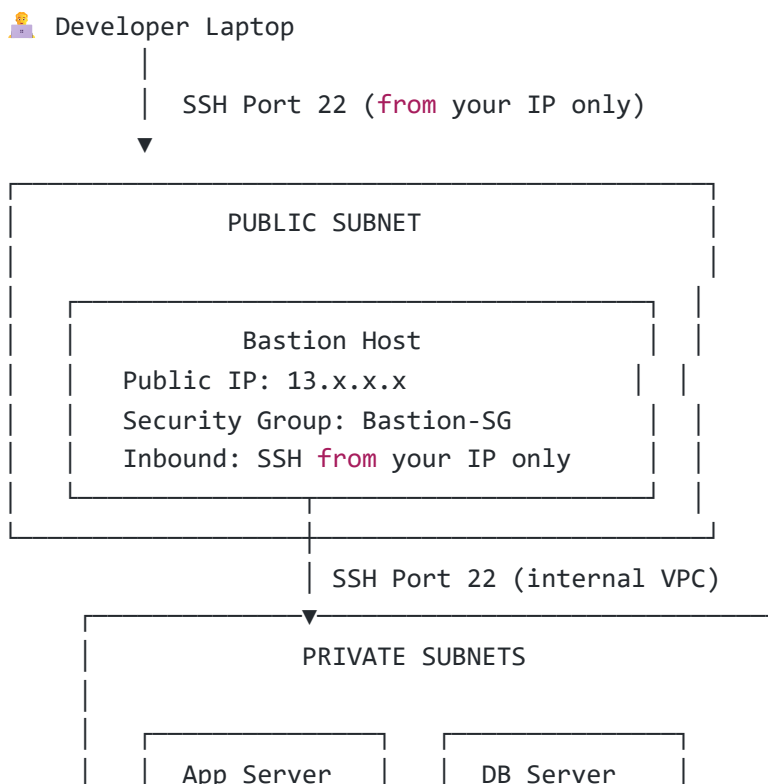
Hop 2: From inside Bastion, connect to private instance

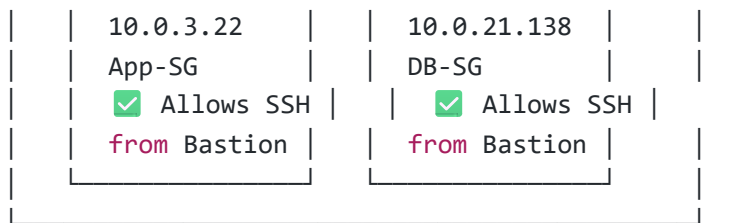
```
ssh -i mykey.pem ubuntu@10.0.3.22
```

Method 2 – Single command using ProxyJump (-J flag)

```
ssh -i mykey.pem -J ubuntu@<bastion-public-ip> ubuntu@10.0.3.22
```

Bastion Host Access Diagram





7. Route Tables

Definition

A **Route Table** is a set of rules — called routes — that tells your VPC where to send network traffic based on its destination. Every subnet in your VPC must be associated with exactly one route table, which governs where traffic from that subnet gets forwarded.

Think of route tables like a GPS system for your network packets. Each packet has a destination IP, and the route table is the map that says "to reach that destination, go through this gateway."

How AWS Picks a Route

When traffic leaves a subnet, AWS checks the route table and picks the route with the **most specific match** (longest prefix):

Traffic going to 10.0.3.55 — which route wins?

Route 1: 10.0.0.0/16 → local (matches — 16 bits specific)

Route 2: 0.0.0.0/0 → IGW (matches — 0 bits specific)

Winner: Route 1 (10.0.0.0/16) — more specific = longer prefix

Result: Traffic stays inside the VPC 

Three Route Tables in This Architecture

Public Route Table

Destination	Target	
10.0.0.0/16	local	← Internal VPC traffic stays inside
0.0.0.0/0	igw-xxxxxxx	← Everything else goes to internet

Associated with: web-public-subnet-1a , web-public-subnet-1b

Private App Route Table

Destination	Target	
10.0.0.0/16	local	← Internal VPC traffic stays inside
0.0.0.0/0	nat-xxxxxxx	← Outbound internet goes via NAT GW

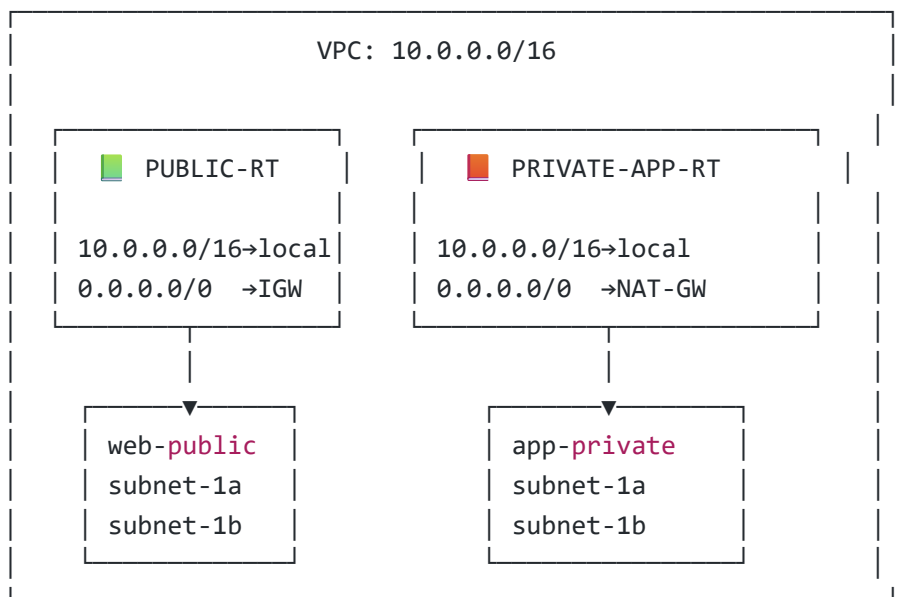
Associated with: app-private-subnet-1a , app-private-subnet-1b

Database Route Table

Destination	Target	
10.0.0.0/16	local	← Internal VPC traffic stays inside
0.0.0.0/0	nat-xxxxxxx	← Outbound internet goes via NAT GW

Associated with: data-private-subnet-1a , data-private-subnet-1b

Route Table Visual Map



The Local Route — Always Present

Every route table contains a `local` route for the VPC CIDR block. This route:

- Cannot be deleted or modified
- Allows all subnets within the VPC to communicate with each other freely
- Does not go through any gateway — traffic stays within AWS's internal fabric

8. Security Groups

Definition

A **Security Group** is a virtual firewall that controls what traffic is allowed to reach (inbound) and leave (outbound) your EC2 instances. It operates at the **instance level**, meaning each instance can have different security groups applied to it.

Unlike traditional firewalls that work at the network boundary, Security Groups attach directly to individual instances — giving you precise per-resource control.

Core Characteristics

- **Stateful** — if an inbound connection is allowed, the response traffic is automatically permitted without needing a separate outbound rule. This is different from NACLs which are stateless.
- **Allow-only rules** — you can only create ALLOW rules. There is no explicit DENY — anything not explicitly allowed is automatically blocked.
- **Reference other Security Groups as sources** — instead of specifying an IP range, you can say "allow traffic from any instance that has Security Group X." This is the most powerful feature for layered architectures.
- Applied at the **Elastic Network Interface (ENI)** level
- Rule changes apply **immediately** — no restart required

Stateful vs Stateless (Quick Clarification)

Stateful (Security Groups):

Inbound rule: ALLOW port 80 **from** 0.0.0.0/0

→ Response traffic (outbound) is automatically allowed 

→ **No** extra outbound rule needed

Stateless (NACLs – Network Access Control Lists):

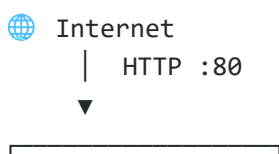
Inbound rule: ALLOW port 80

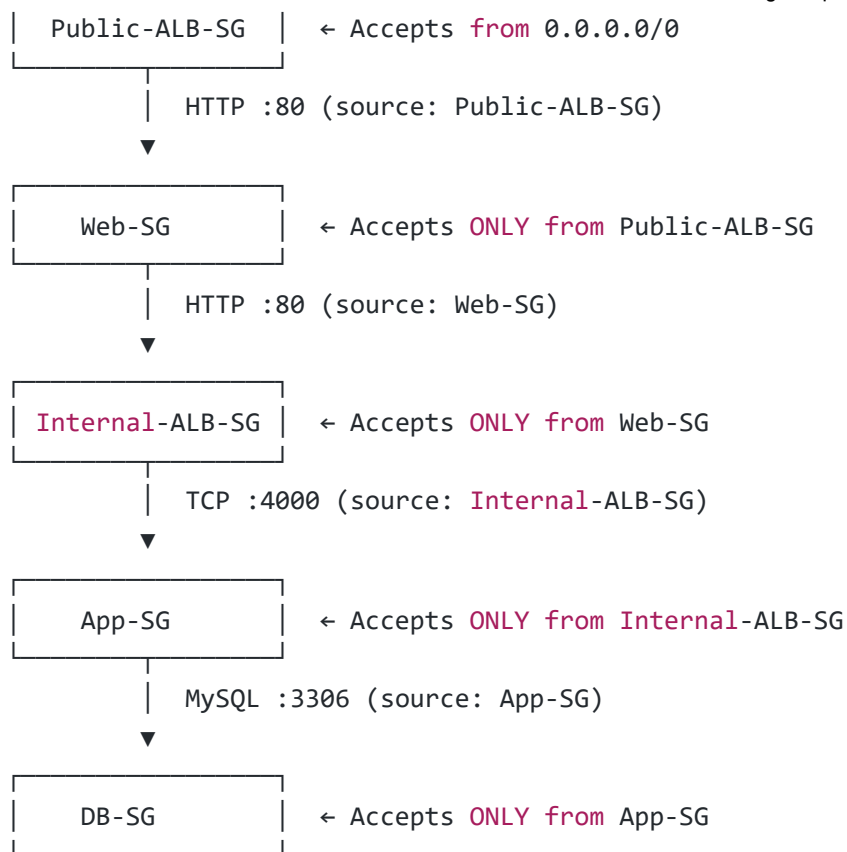
→ You must **ALSO** add an outbound rule **to** allow the response 

→ Both directions must be explicitly configured

Security Group Chaining

This is the architecture that makes AWS networking truly secure. Each layer only accepts traffic from the layer directly above it, using SG references instead of IP ranges.





This creates a **zero-trust chain** — each layer is completely isolated and can only be reached from its direct upstream component.

9. Production Task — Three-Tier Architecture

What We're Building

A fully production-grade, highly available three-tier web application deployed across two Availability Zones with auto-scaling at every tier.

Architecture Stack:

- └─  Web Tier → NGINX serving React frontend + reverse proxy
- └─  App Tier → Node.js REST API running on port 4000
- └─  DB Tier → MySQL database instance (isolated)

Infrastructure Details

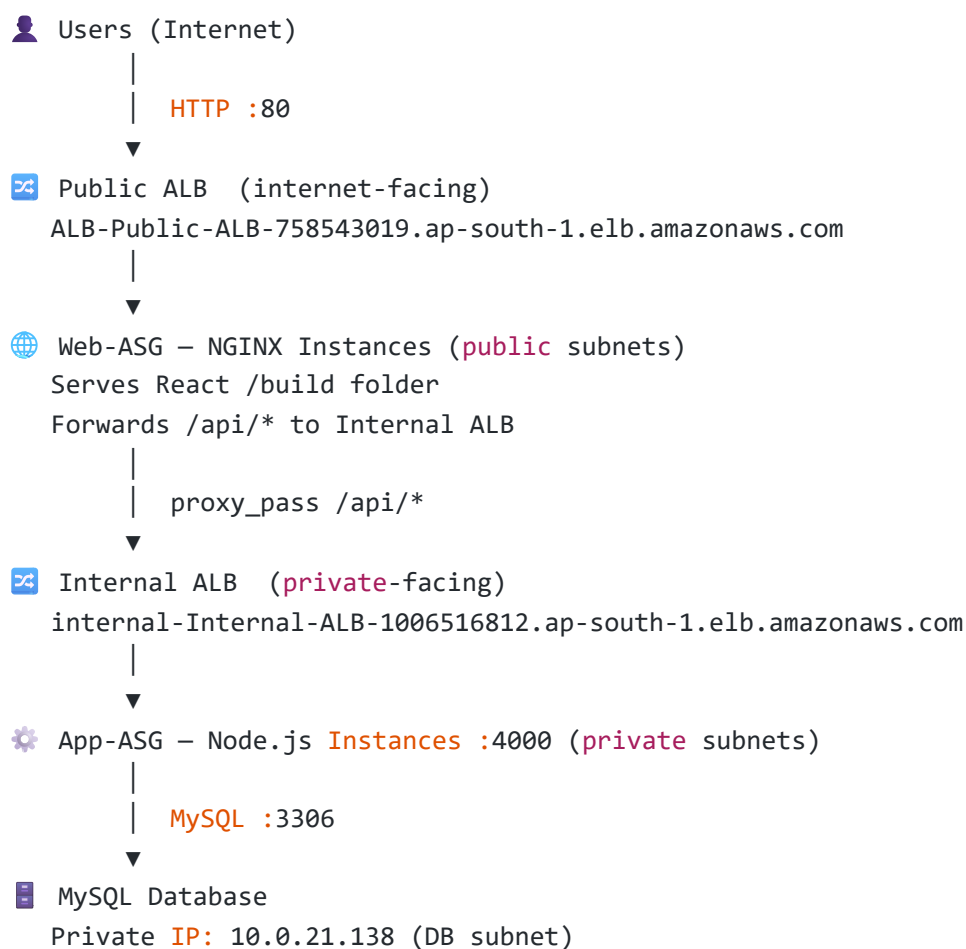
Component	Value
VPC CIDR	10.0.0.0/16
Region	ap-south-1 (Mumbai)

Component	Value
Availability Zones	ap-south-1a and ap-south-1b
Public ALB DNS	ALB-Public-ALB-758543019.ap-south-1.elb.amazonaws.com
Internal ALB DNS	internal-Internal-ALB-1006516812.ap-south-1.elb.amazonaws.com

Subnet Layout

Subnet Name	Category	AZ	CIDR
web-public-subnet-1a	 Public	ap-south-1a	10.0.1.0/24
web-public-subnet-1b	 Public	ap-south-1b	10.0.2.0/24
app-private-subnet-1a	 Private App	ap-south-1a	10.0.3.0/24
app-private-subnet-1b	 Private App	ap-south-1b	10.0.4.0/24
data-private-subnet-1a	 Private DB	ap-south-1a	10.0.5.0/24
data-private-subnet-1b	 Private DB	ap-south-1b	10.0.6.0/24

End-to-End Request Flow



Step-by-Step Implementation

✓ Step 1 — Create the VPC

AWS Console → VPC → Your VPCs → Create VPC

Setting	Value
Name Tag	Production-VPC
IPv4 CIDR	10.0.0.0/16
Tenancy	Default

✓ Step 2 — Create and Attach Internet Gateway

VPC → Internet Gateways → Create Internet Gateway

Setting	Value
Name Tag	Production-IGW

After creation: Actions → Attach to VPC → Select Production-VPC → Attach

Without attaching the IGW, your public subnets cannot reach the internet even with the right route table entries.

✓ Step 3 — Create Public Subnets

VPC → Subnets → Create Subnet → Select Production-VPC

Name	AZ	CIDR
web-public-subnet-1a	ap-south-1a	10.0.1.0/24
web-public-subnet-1b	ap-south-1b	10.0.2.0/24

After creating each: Select subnet → Actions → Edit Subnet Settings → Enable Auto-assign Public IPv4 ✓

✓ Step 4 — Create App Private Subnets

Name	AZ	CIDR
app-private-subnet-1a	ap-south-1a	10.0.3.0/24
app-private-subnet-1b	ap-south-1b	10.0.4.0/24

Do NOT enable auto-assign public IP here. These stay private.

✅ Step 5 — Create Database Private Subnets

Name	AZ	CIDR
data-private-subnet-1a	ap-south-1a	10.0.5.0/24
data-private-subnet-1b	ap-south-1b	10.0.6.0/24

✅ Step 6 — Create NAT Gateway

VPC → NAT Gateways → Create NAT Gateway

Setting	Value
Name	Production-NAT
Subnet	web-public-subnet-1a ← must be a public subnet
Elastic IP	Click "Allocate Elastic IP"

⌚ Wait for NAT Gateway status to show **Available** before moving to route tables — usually takes 1-2 minutes.

Why this matters: Private subnet instances (app servers, database) need outbound internet access for:

```
apt update      → OS security patches
npm install     → Node.js dependencies
git clone       → Fetching application code
```

✅ Step 7 — Configure Route Tables

VPC → Route Tables → Create Route Table

🟢 Public Route Table

Setting	Value
Name	Public-RT
VPC	Production-VPC

Routes to add:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-IGW

Subnet Associations: web-public-subnet-1a , web-public-subnet-1b

Private App Route Table

Setting	Value
Name	Private-App-RT
VPC	Production-VPC

Routes to add:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-NAT

Subnet Associations: app-private-subnet-1a , app-private-subnet-1b

Database Route Table

Setting	Value
Name	Database-RT
VPC	Production-VPC

Routes to add:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-NAT

Subnet Associations: data-private-subnet-1a , data-private-subnet-1b

✅ Step 8 — Configure Security Groups

EC2 → Security Groups → Create Security Group

🔒 Bastion-SG

Rule	Protocol	Port	Source
Inbound — SSH	TCP	22	0.0.0.0/0

🔒 Public-ALB-SG

Rule	Protocol	Port	Source
Inbound — HTTP	TCP	80	0.0.0.0/0

🔒 Web-SG

Rule	Protocol	Port	Source
Inbound — HTTP	TCP	80	Public-ALB-SG
Inbound — SSH	TCP	22	Bastion-SG

🔒 Internal-ALB-SG

Rule	Protocol	Port	Source
Inbound — HTTP	TCP	80	Web-SG

🔒 App-SG

Rule	Protocol	Port	Source
Inbound — Custom TCP	TCP	4000	Internal-ALB-SG
Inbound — SSH	TCP	22	Bastion-SG

DB-SG

Rule	Protocol	Port	Source
Inbound — MySQL	TCP	3306	App-SG
Inbound — SSH	TCP	22	Bastion-SG

Security Group Assignment Map

Resource	Uses Security Group
Bastion Host	→ Bastion-SG
Public ALB	→ Public-ALB-SG
Web Instances	→ Web-SG
Internal ALB	→ Internal-ALB-SG
App Instances	→ App-SG
MySQL Database	→ DB-SG

✅ Step 9 — Launch Bastion Host

EC2 → Launch Instance

Setting	Value
Name	Bastion-Host
AMI	Ubuntu Server 22.04 LTS
Instance Type	t2.micro
Subnet	web-public-subnet-1a
Auto-assign Public IP	Enabled 
Security Group	Bastion-SG

✅ Step 10 — Create Internal ALB

EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Setting	Value
Name	Internal-ALB
Scheme	Internal
Subnets	app-private-subnet-1a, app-private-subnet-1b
Security Group	Internal-ALB-SG

 Step 11 — Create App Target Group

EC2 → Target Groups → Create Target Group

Setting	Value
Name	App-TG
Protocol	HTTP
Port	4000
Target Type	Instance
Health Check Protocol	HTTP
Health Check Path	/health

 Step 12 — Launch Database EC2 Instance

EC2 → Launch Instance

Setting	Value
Name	DB-Instance
AMI	Ubuntu Server 22.04 LTS
Subnet	data-private-subnet-1a
Auto-assign Public IP	Disabled 
Security Group	DB-SG

✓ Step 13 — Install and Configure MySQL

SSH into DB instance through Bastion, then:

```
# System update
sudo apt update && sudo apt upgrade -y

# Install MySQL Server
sudo apt install mysql-server -y

# Enter MySQL shell
sudo mysql
```

Inside MySQL:

```
-- Create the application database
CREATE DATABASE webappdb;

-- Create dedicated app user (not using root)
CREATE USER 'appuser'@'%' IDENTIFIED BY 'YourStrongPassword123!';

-- Grant access only to the app database
GRANT ALL PRIVILEGES ON webappdb.* TO 'appuser'@'%';

-- Apply privilege changes
FLUSH PRIVILEGES;

EXIT;
```

Allow remote connections from App tier:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

Find and change:

```
# Before (only accepts local connections):
bind-address = 127.0.0.1

# After (accepts from all interfaces – App-SG restricts who can actually reach it):
bind-address = 0.0.0.0
```

Restart MySQL:

```
sudo systemctl restart mysql
sudo systemctl enable mysql
```

✓ Step 14 — Launch App Tier Test Instance

EC2 → Launch Instance

Setting	Value
Name	App-Test-Instance
AMI	Ubuntu Server 22.04 LTS
Subnet	app-private-subnet-1a
Auto-assign Public IP	Disabled ✗
Security Group	App-SG

✓ Step 15 — Install Node.js and PM2

SSH in via Bastion, then:

```
# System update
sudo apt update && sudo apt upgrade -y

# Add Node.js 16 package source
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash -

# Install Node.js and Git
sudo apt install -y nodejs git

# Install PM2 globally (keeps Node.js app alive after terminal closes)
sudo npm install -g pm2
```

What is PM2? PM2 is a process manager for Node.js. It keeps your app running in the background, restarts it if it crashes, and can be configured to auto-start on server reboot. Essential for production Node.js deployments.

✓ Step 16 — Clone Application Code

```
git clone https://github.com/Asadkhanrtx/aws-three-tier-web-architecture-workshop.git
```

✓ Step 17 — Set Database Configuration

```
nano aws-three-tier-web-architecture-workshop/application-code/app-tier/DbConfig.js
```

```
module.exports = Object.freeze({  
  DB_HOST      : '10.0.21.138',           // Private IP of your MySQL instance  
  DB_USER      : 'appuser',  
  DB_PWD       : 'YourStrongPassword123!',  
  DB_DATABASE  : 'webappdb'  
});
```

✅ Step 18 — Start the App Tier

```
cd aws-three-tier-web-architecture-workshop/application-code/app-tier
```

```
# Install all Node.js dependencies  
npm install
```

```
# Launch app with PM2  
pm2 start index.js --name app-tier
```

```
# Save process list so it survives reboots  
pm2 save
```

Verify the app is running correctly:

```
curl http://localhost:4000/health
```

Expected output:

```
"This is the health check"
```

✅ Step 19 — Register App Instance to Target Group

1. EC2 → Target Groups → App-TG → Register Targets
2. Select the App Test Instance
3. Click **Include as pending below** → Register pending targets
4. Wait for health status to show ✅ Healthy

✅ Step 20 — Create Public ALB

EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Setting	Value
Name	Public-ALB
Scheme	Internet-facing
Subnets	web-public-subnet-1a, web-public-subnet-1b
Security Group	Public-ALB-SG
Listener	HTTP :80 → forward to Web-TG

 Step 21 — Create Web Target Group

EC2 → Target Groups → Create Target Group

Setting	Value
Name	Web-TG
Protocol	HTTP
Port	80
Target Type	Instance
Health Check Path	/

 Step 22 — Launch Web Tier Test Instance

EC2 → Launch Instance

Setting	Value
Name	Web-Test-Instance
AMI	Ubuntu Server 22.04 LTS
Subnet	web-public-subnet-1a
Auto-assign Public IP	Enabled 
Security Group	Web-SG

✓ Step 23 — Install NGINX and Node.js on Web Tier

SSH directly into web instance (it has a public IP):

```
# System update
sudo apt update && sudo apt upgrade -y

# Add Node.js 16 repository
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash -

# Install Node.js, NGINX, and Git together
sudo apt install nodejs nginx git -y
```

Clone the repository:

```
git clone https://github.com/Asadkhanrtx/aws-three-tier-web-architecture-workshop.git
```

✓ Step 24 — Build the React Frontend

```
cd aws-three-tier-web-architecture-workshop/application-code/web-tier

# Install React dependencies
npm install

# Build production-optimized bundle
npm run build
```

This generates a `/build` directory containing compiled, minified static files that NGINX will serve.

✓ Step 25 — Configure NGINX

```
sudo nano /etc/nginx/sites-available/default
```

Replace the entire file with:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # Health check endpoint for ALB target group checks
    location /health {
        default_type text/html;
    }
}
```

```

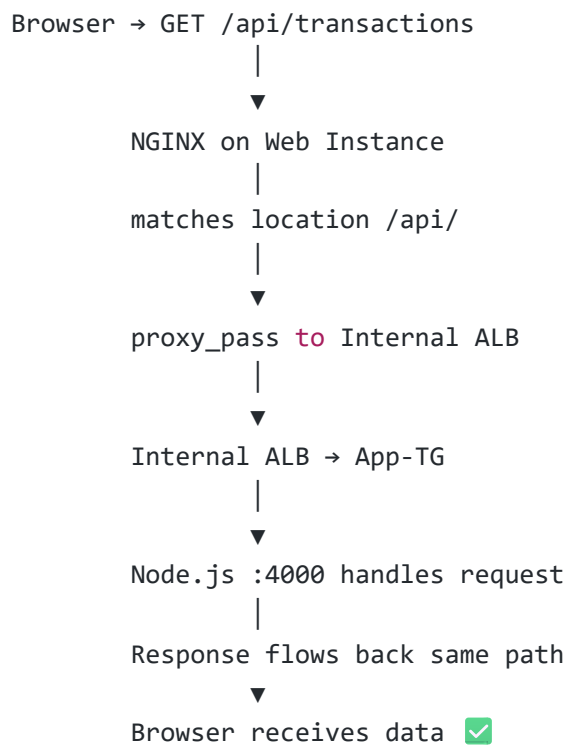
    return 200 "<!DOCTYPE html><p>Web Tier Health Check</p>\n";
}

# Serve compiled React frontend
location / {
    root /home/ubuntu/aws-three-tier-web-architecture-workshop/application-code/web-tier/b
    index index.html index.htm;
    try_files $uri /index.html;
}

# Reverse proxy: all /api/ requests forwarded to Internal ALB → App Tier
location /api/ {
    proxy_pass http://internal-Internal-ALB-1006516812.ap-south-1.elb.amazonaws.com;
}
}

```

How NGINX Bridges Web and App Tiers



✓ Step 26 — Test and Reload NGINX

```

# Validate NGINX config before applying
sudo nginx -t

# Restart to apply changes
sudo systemctl restart nginx

```

```
# Enable auto-start on reboot
sudo systemctl enable nginx
```

✅ Step 27 — Register Web Instance to Web Target Group

1. EC2 → Target Groups → Web-TG → Register Targets
2. Select Web Test Instance → Register
3. Wait for health status: ✅ Healthy

✅ Step 28 — End-to-End Testing

Test the frontend:

<http://ALB-Public-ALB-758543019.ap-south-1.elb.amazonaws.com>

Test API via web tier:

```
curl http://localhost/api/health
```

Expected:

```
"This is the health check"
```

Full stack test: Use the DB Demo page in the React app to verify read/write to MySQL through the full three-tier chain.

✅ Step 29 — Create AMIs for Auto Scaling

Capture the current state of your tested instances as reusable images.

EC2 → Instances → Select Instance → Actions → Image and Templates → Create Image

Instance	AMI Name	Note
Web-Test-Instance	ubuntu-web-tier-ami-v1	Enable reboot for clean filesystem
App-Test-Instance	ubuntu-app-tier-ami-v1	Enable reboot for clean filesystem

Enabling "Reboot instance" before snapshot ensures all data is written to disk properly, giving you a consistent AMI.

✅ Step 30 — Create Launch Templates

EC2 → Launch Templates → Create Launch Template

📋 Web Launch Template

Setting	Value
Name	Web-LT
AMI	ubuntu-web-tier-ami-v1
Security Group	Web-SG

📋 App Launch Template

Setting	Value
Name	App-LT
AMI	ubuntu-app-tier-ami-v1
Security Group	App-SG

✅ Step 31 — Create Auto Scaling Groups

EC2 → Auto Scaling Groups → Create Auto Scaling Group

⚙️ Web-ASG

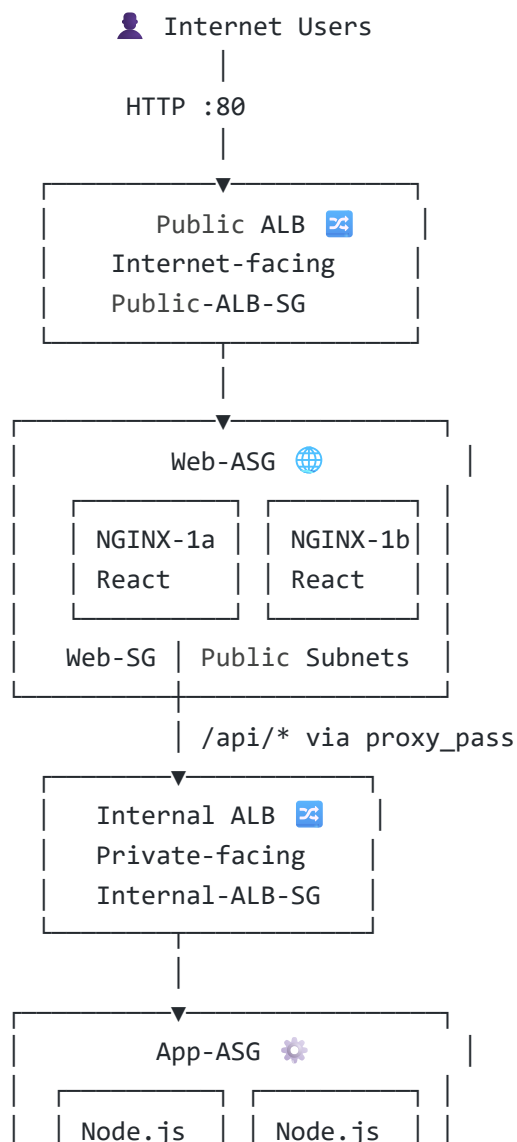
Setting	Value
Name	Web-ASG
Launch Template	Web-LT
Subnets	web-public-subnet-1a, web-public-subnet-1b
Target Group	Web-TG
Desired	2
Minimum	2
Maximum	4

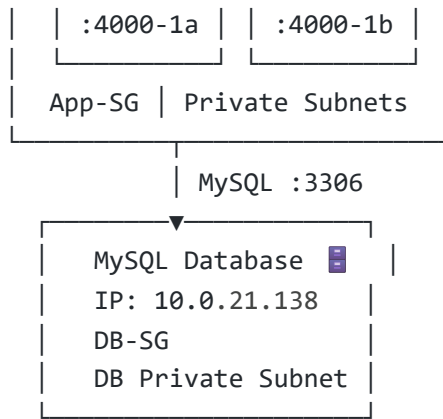
⚙️ App-ASG

Setting	Value
Name	App-ASG
Launch Template	App-LT
Subnets	app-private-subnet-1a, app-private-subnet-1b
Target Group	App-TG
Desired	2
Minimum	2
Maximum	4

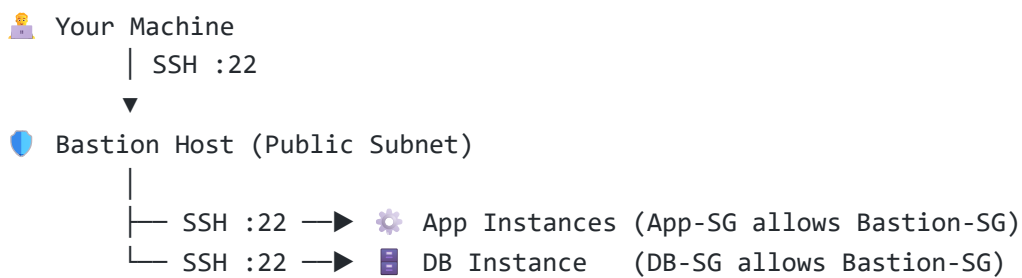
Final Architecture Summary

Complete Traffic Flow





SSH Access Path



Auto Scaling Summary

Web-ASG	
Template	: Web-LT
AMI	: web-tier-ami-v1
SG	: Web-SG
Subnets	: public-1a, 1b
Capacity	: 2 min / 4 max
Target	: Web-TG

App-ASG	
Template	: App-LT
AMI	: app-tier-ami-v1
SG	: App-SG
Subnets	: private-1a, 1b
Capacity	: 2 min / 4 max
Target	: App-TG



Production Benefits of This Architecture

 Benefit	 How It's Achieved
High Availability	Every tier deployed across two Availability Zones
Fault Tolerance	Auto Scaling Groups replace failed instances automatically
Backend Security	App and DB servers have no public IP, not reachable from internet
Elastic Scalability	ASGs scale from 2 to 4 instances based on actual load
Controlled SSH Access	Single audited entry point via Bastion Host
Clean Separation of Concerns	NGINX handles frontend; Node.js handles API; MySQL handles data
Database Isolation	DB subnet only reachable from App-SG on port 3306
Dual Load Balancing	Two ALBs distribute load at web tier and app tier independently
Future-Proof IP Design	/16 CIDR gives room for 65,536 IPs and hundreds of subnets
No Direct DB Internet Exposure	DB subnet has no IGW route — outbound-only via NAT